



University
of Exeter

COMM117

Large Language Models and Application

Previous Lecture

Transformer!

- **Sequential compute → more parallel training**

RNN: must process tokens **one-by-one**.

Transformer/LLM: in each layer, it computes **attention** for all positions **in parallel** (during training).

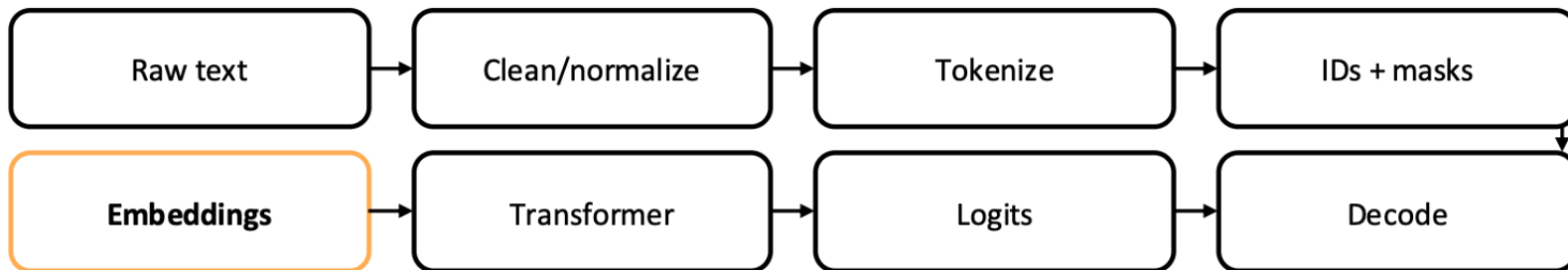
- **Long-range dependencies → direct connections via attention**

RNN: information must travel through many steps (weak signal over long distance).

LLM: a token can attend **directly to a far-away token** in one operation.

Previous Lecture

End-to-end NLP / LLM pipeline



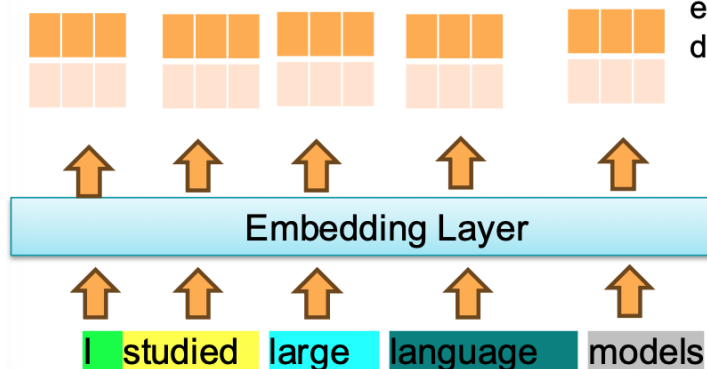
Previous Lecture

Embedding in LLM

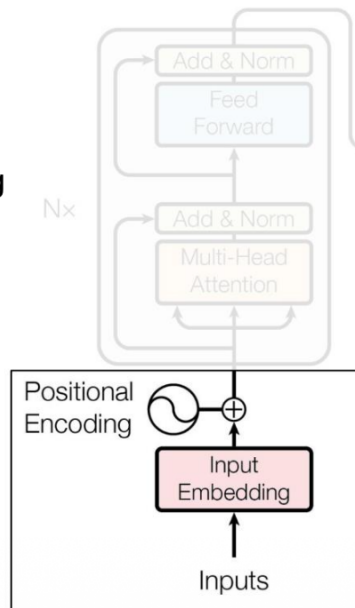
$$E \in \mathbb{R}^{\{|V| \times d\}}$$

Learnable Vectors

D is model
embedding
dimension



Generate Input Embeddings



Previous Lecture

Attention

To find the attention of a target word relative to input embeddings:

- Use the **Query (Q)** of the **target word**.
- Use the **Key (K)** of the **input word**.
- Compute a **matching score (attention weight)** from Q and K.
- Use these scores as weights to sum the Value (V) vectors.

Previous Lecture

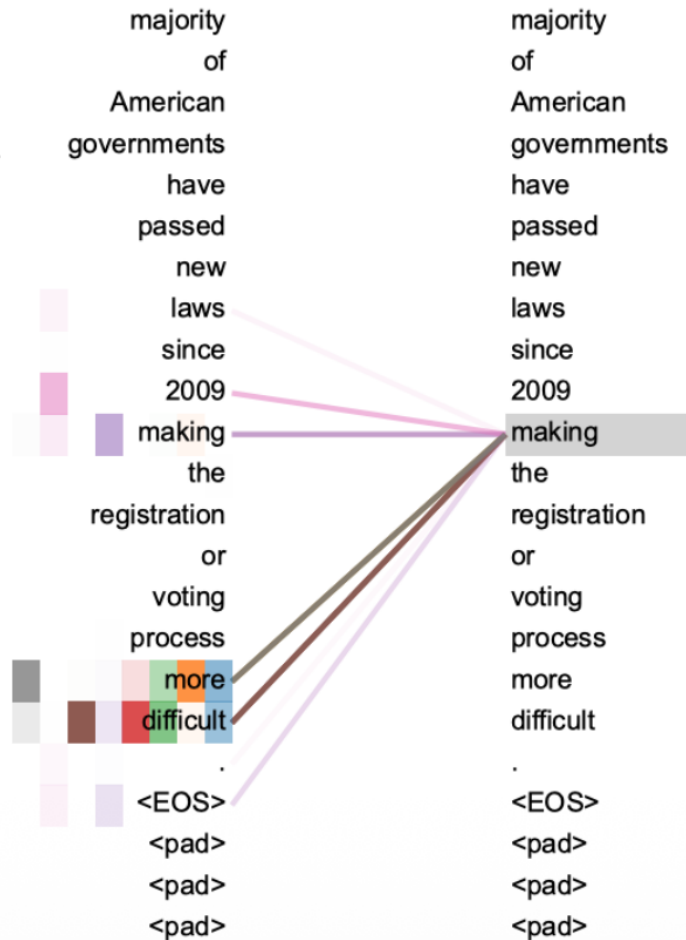
Attention

To find the attention of a target word relative to input embeddings:

- Use the **Query (Q)** of the **target word**.
- Use the **Key (K)** of the **input word**.
- Compute a **matching score (attention weight)** from Q and K.
- Use these scores as weights to sum the Value (V) vectors.

Self-attention

- Input: token embeddings
 $X \in R^{n \times d}$
- **Project to Q,K,V with learned matrices**
- Each position can attend to all others (global receptive field)

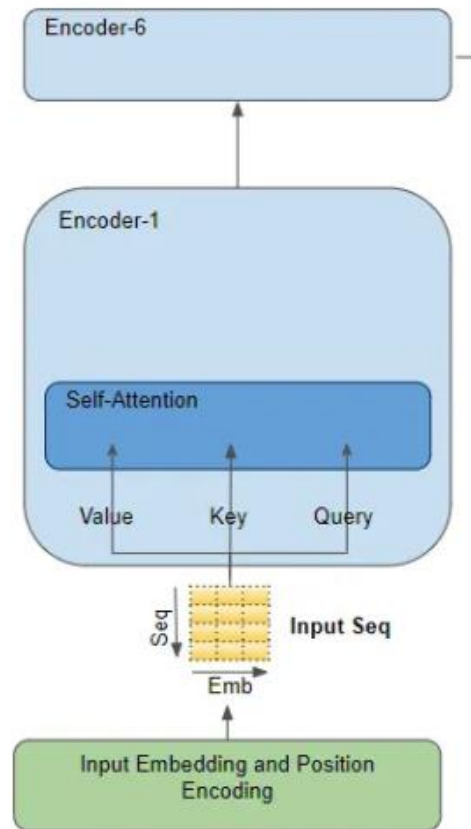
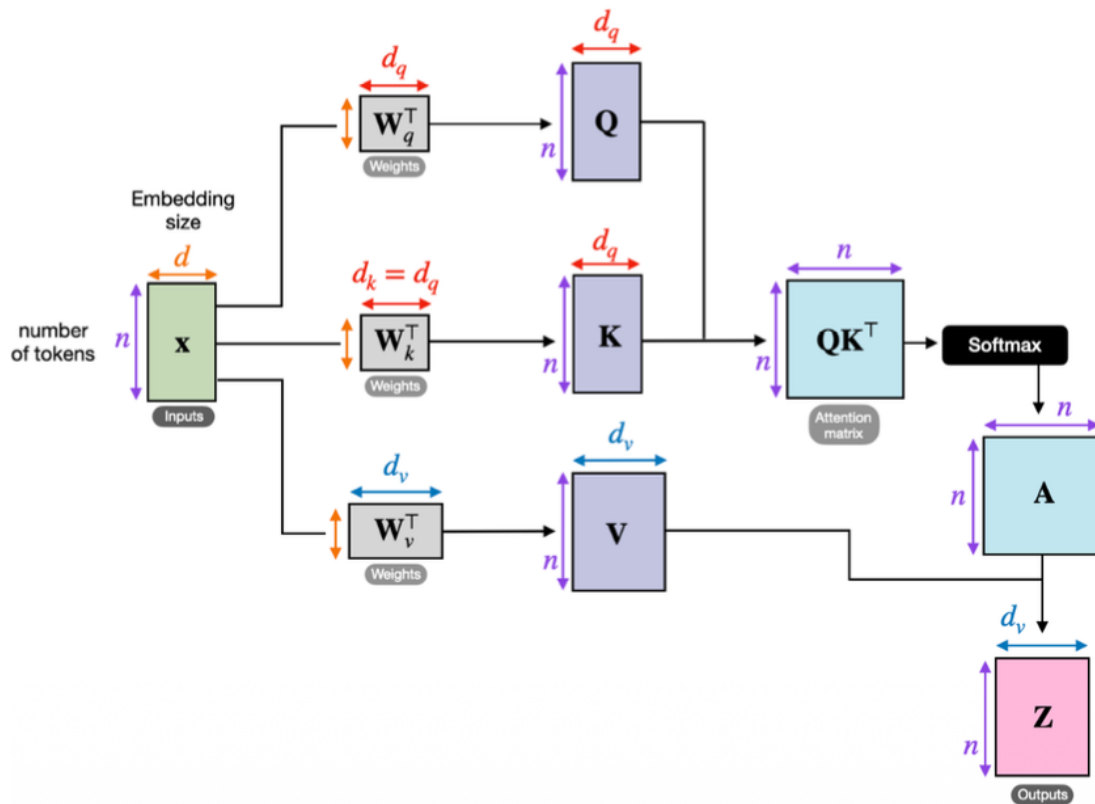


Introduction Today

- Multi-head Attention
- Transformer Structure

Residual Connection (Add), LayerNorm,
Feedforward Network(FFN)

Encoder Self-Attention



One-head attention

- Every token forms **one** query vector and compares it to keys in **one** learned similarity space.

$$score(t, s) = \frac{q_t^T k_s}{\sqrt{d_k}}$$

Similarity is computed in a **single projection space** determined by W_Q, W_k

One-head attention

- Every token forms **one** query vector and compares it to keys in **one** learned similarity space.

$$score(t, s) = \frac{q_t^T k_s}{\sqrt{d_k}}$$

Similarity is computed in a **single projection space** determined by W_Q, W_k

However

Language contains **multiple concurrent relations** (syntax, coreference, discourse structure, positional locality, named entity linking, topic).

One head = one similarity space for all relationships

Multi-head = multiple learned retrieval views

- Each head has **its own** W_q W_K W_V
- Heads compute different attention patterns **in parallel**

Parallel specialists-- Typical patterns

- Local **syntactic** links (e.g., determiner → noun, adjective → noun)
- **Long-range** dependencies (e.g., subject–verb agreement across clauses)
- Coreference / **entity tracking** (pronouns → antecedents)
- Delimiter / **structural tokens** (e.g., attending to separators, sentence starts)
- **Copy behaviour** in generation (attend strongly to a specific previous token)

Parallel specialists-- Typical patterns

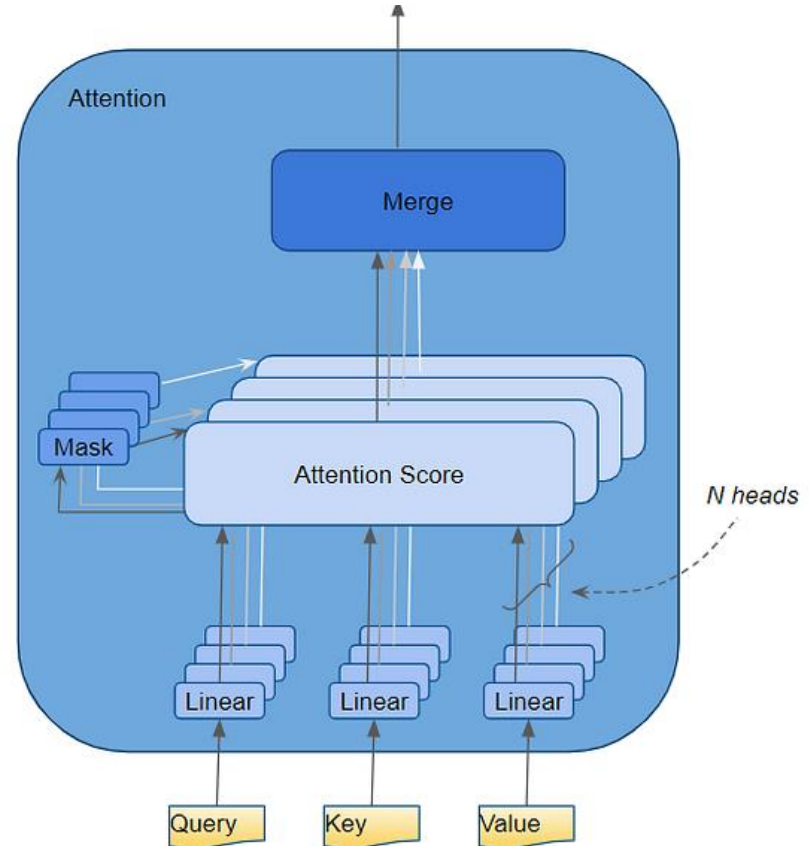
- Local **syntactic** links (e.g., determiner → noun, adjective → noun)
- **Long-range** dependencies (e.g., subject–verb agreement across clauses)
- Coreference / **entity tracking** (pronouns → antecedents)
- Delimiter / **structural tokens** (e.g., attending to separators, sentence starts)
- **Copy behaviour** in generation (attend strongly to a specific previous token)

Good !

- If one head is “busy” capturing **locality**, it may be suboptimal at capturing **long-range links**.
- Multiple heads let the model allocate capacity across patterns.

Multi-head Attention

- Attention module repeats its computations multiple times in **parallel**
- Attention module splits its Query, Key, and Value parameters **N-ways** and **passes each split** independently through a separate Head.



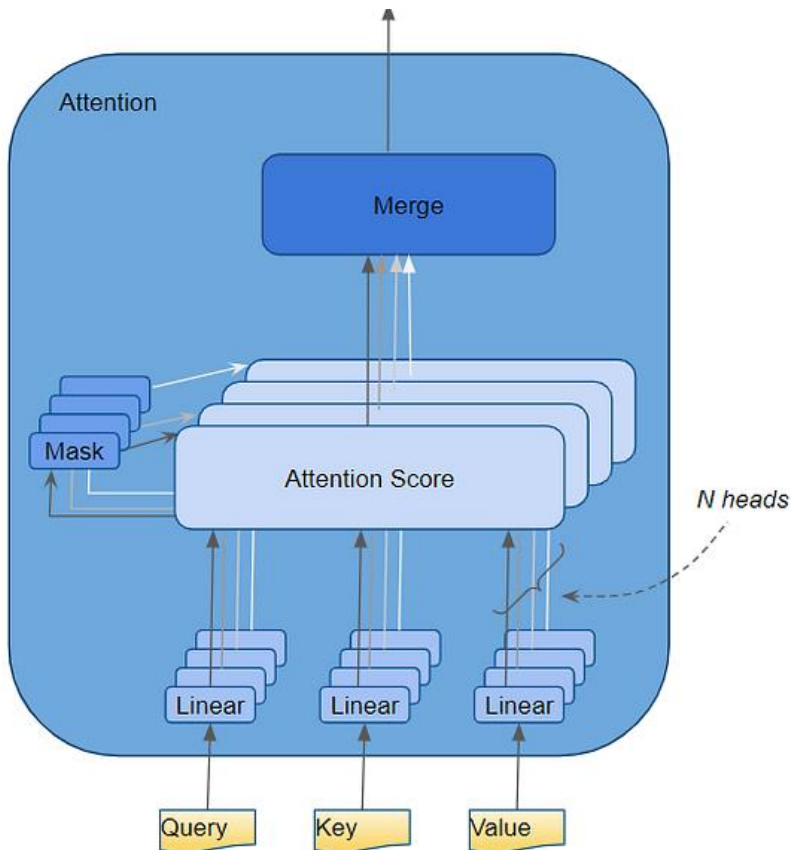
Multi-head Attention

- Increased expressivity without changing sequence length
- Output is a **weighted sum of values**

$$y_t = \sum \alpha v \quad \text{One head}$$

$$y_t = \sum_i^n \alpha_i v_i \quad \text{Multi heads}$$

multiple independently-computed summaries



Multi-head Attention

- “Soft clustering” in multiple subspaces

$$y_t = \sum_i^n \alpha_i v_i$$


Attention weights:

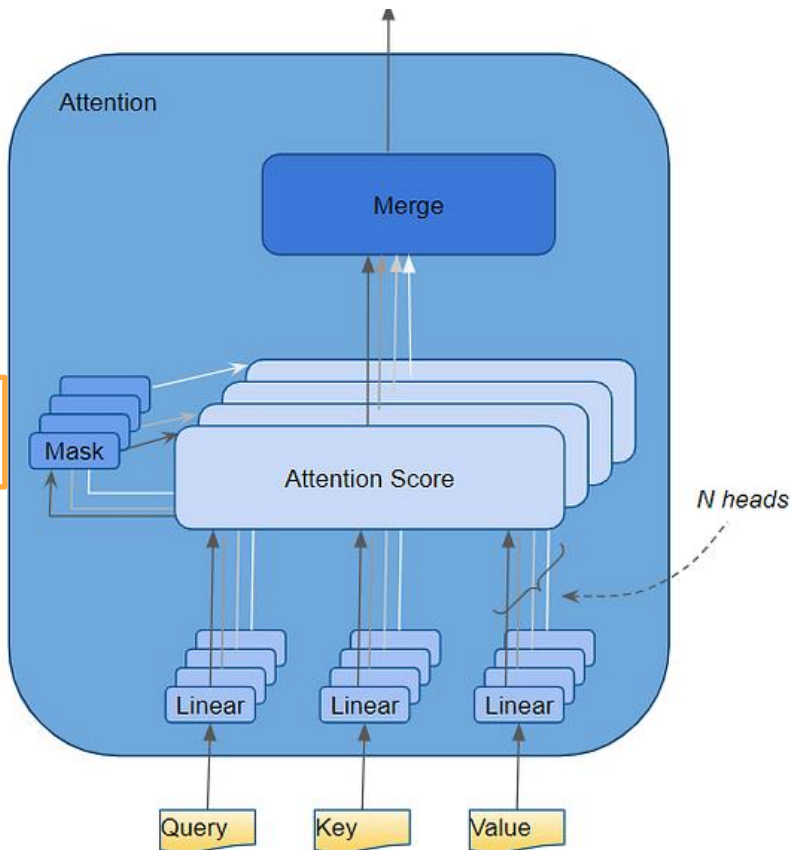
a soft distribution over which tokens matter for token t

- This resembles doing **multiple soft clustering**

Head A clusters by syntactic role

Head B clusters by semantic similarity

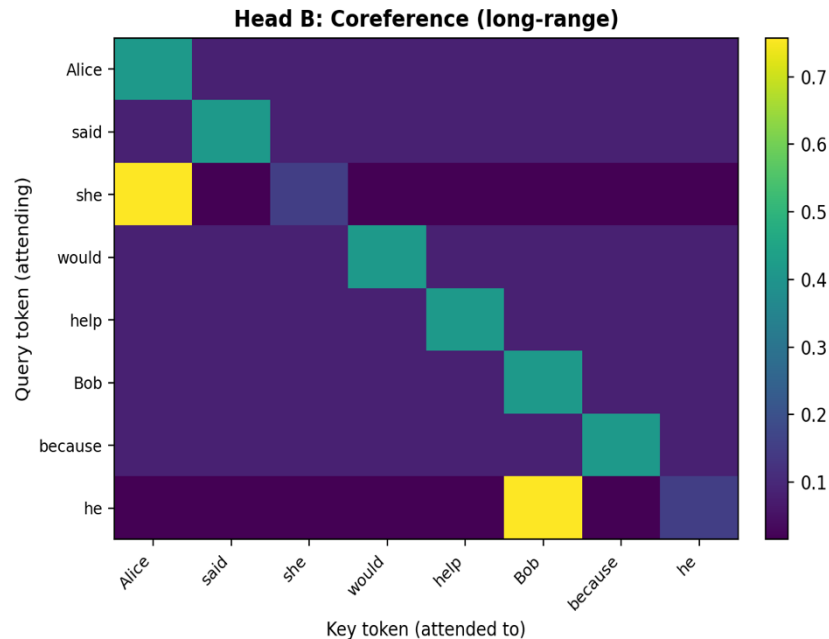
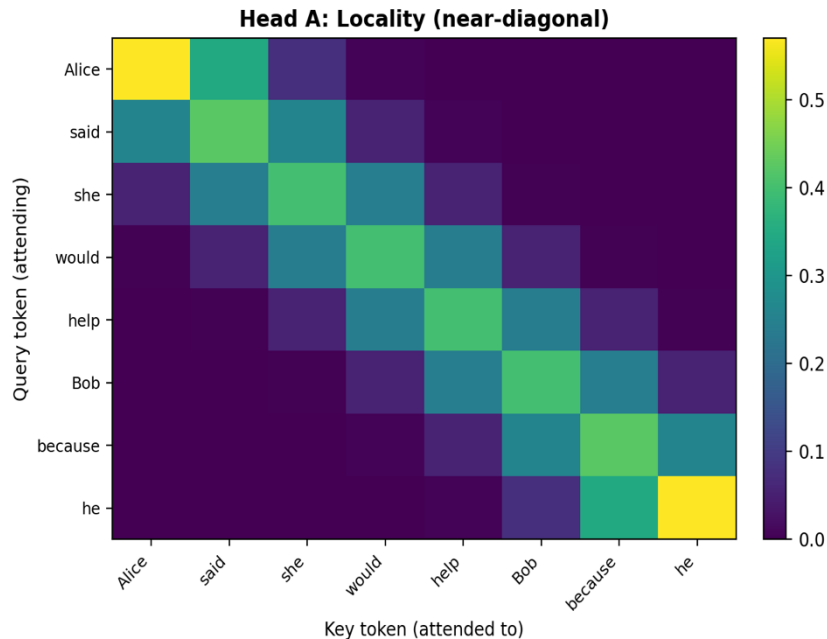
Head C clusters by positional neighbourhood



Different attention patterns

Same sentence, two heads learn different token-to-token alignments:

- Head A concentrates on near-diagonal (local phrase/syntax cues).
- Head B links pronouns to earlier antecedents (long-range coreference).



Multi-head Attention

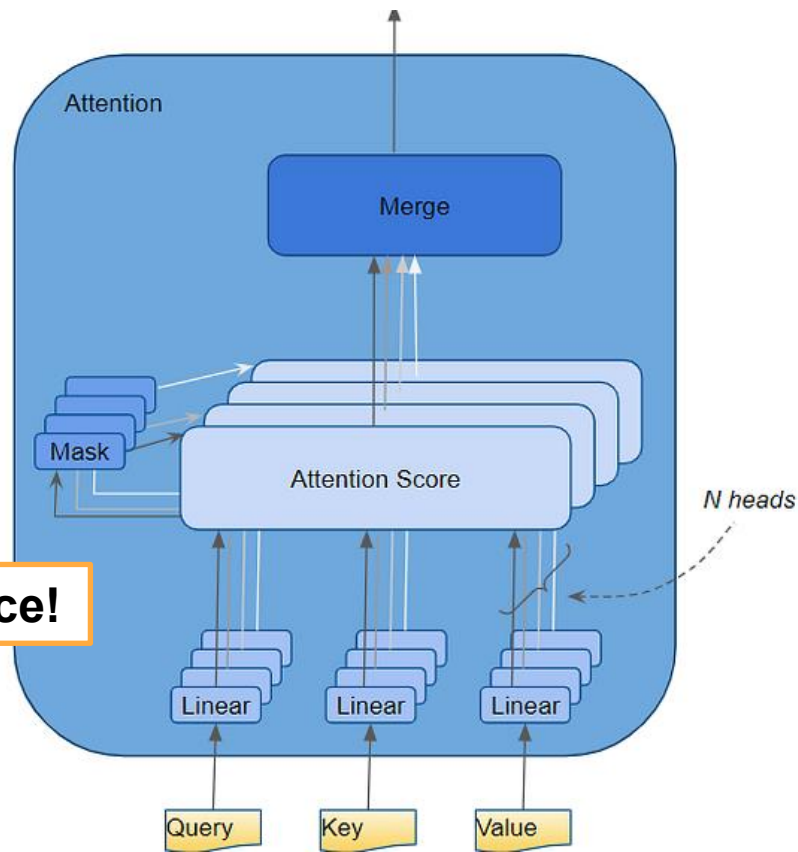
Optimization and feature separation benefits

- Each head works with **lower-dimensional projections** make dot-products and softmax distributions easier to manage.
- Multiple heads reduce the risk that one set of attention weights must serve incompatible objectives.

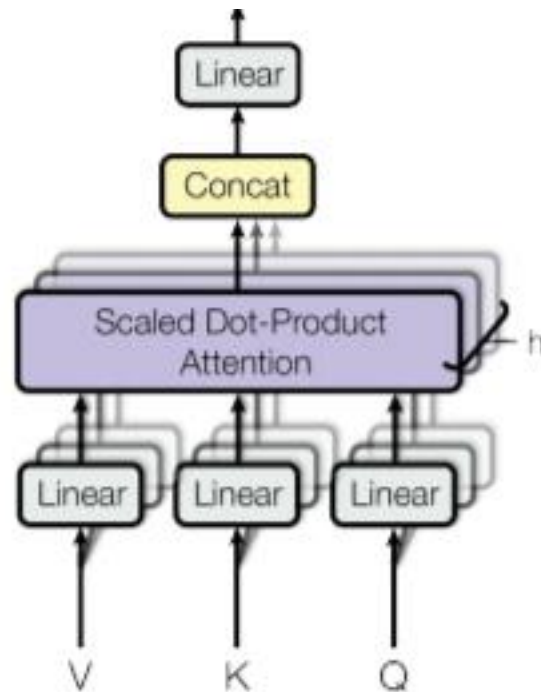
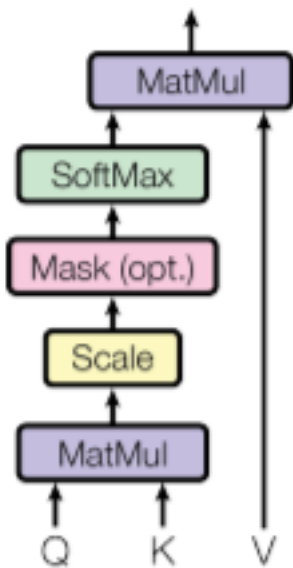
Multi-head Attention

- Remains the same compute order $O(n^2)$

same compute order, better performance!



Multi-head Attention mechanics



$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Multi-head Attention Linear Layers

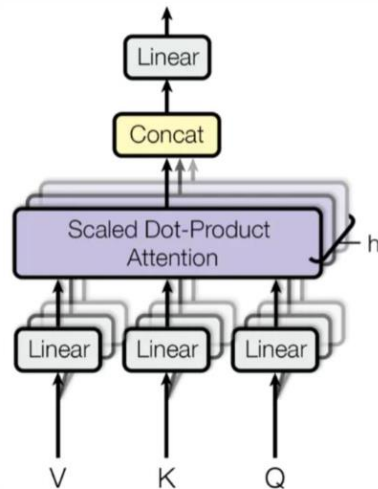
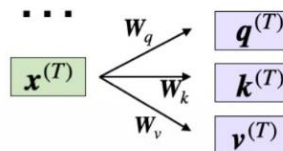
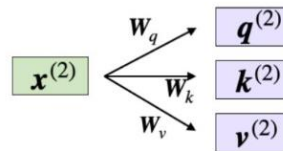
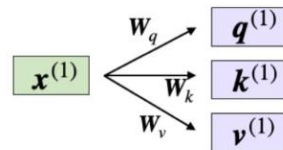
Linear Transformation

- X is projected into multiple smaller-dimensional subspaces using different weight matrices.

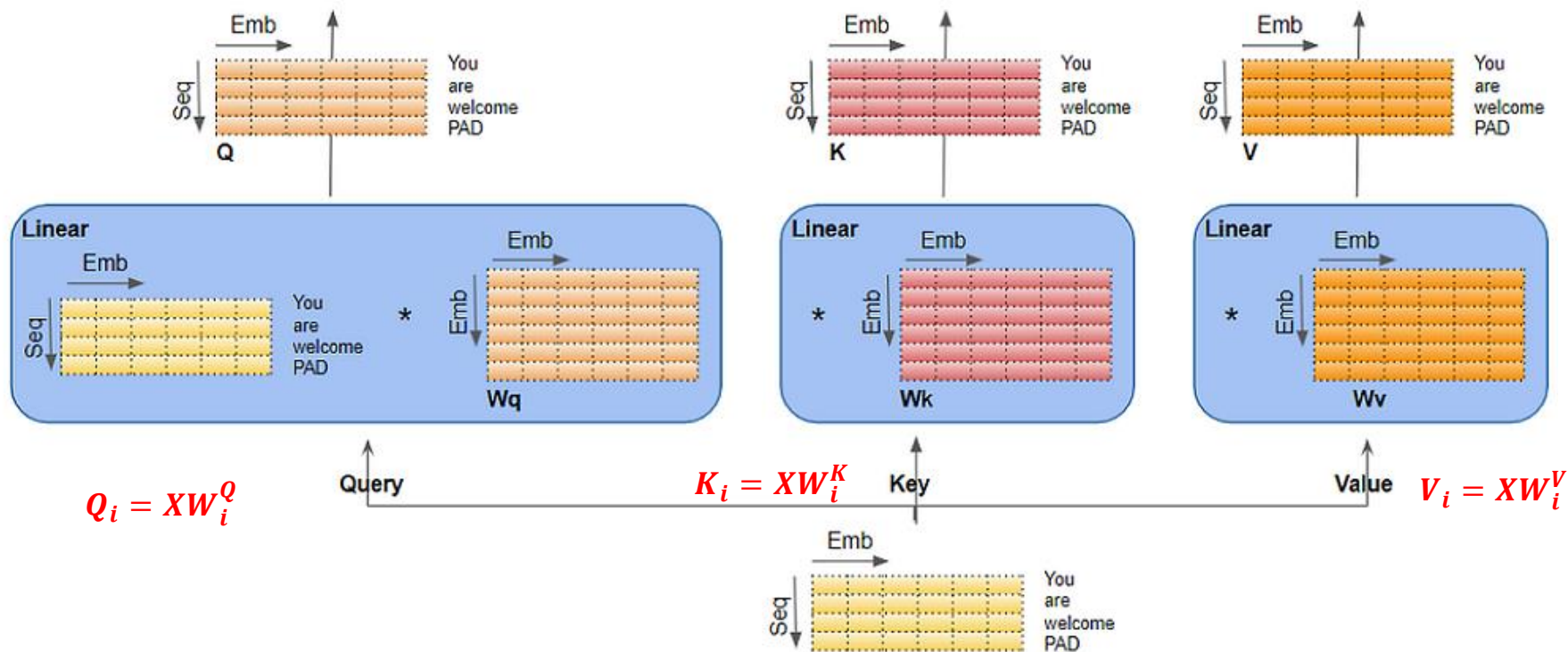
$$Q_i = XW_i^Q$$

$$K_i = XW_i^K$$

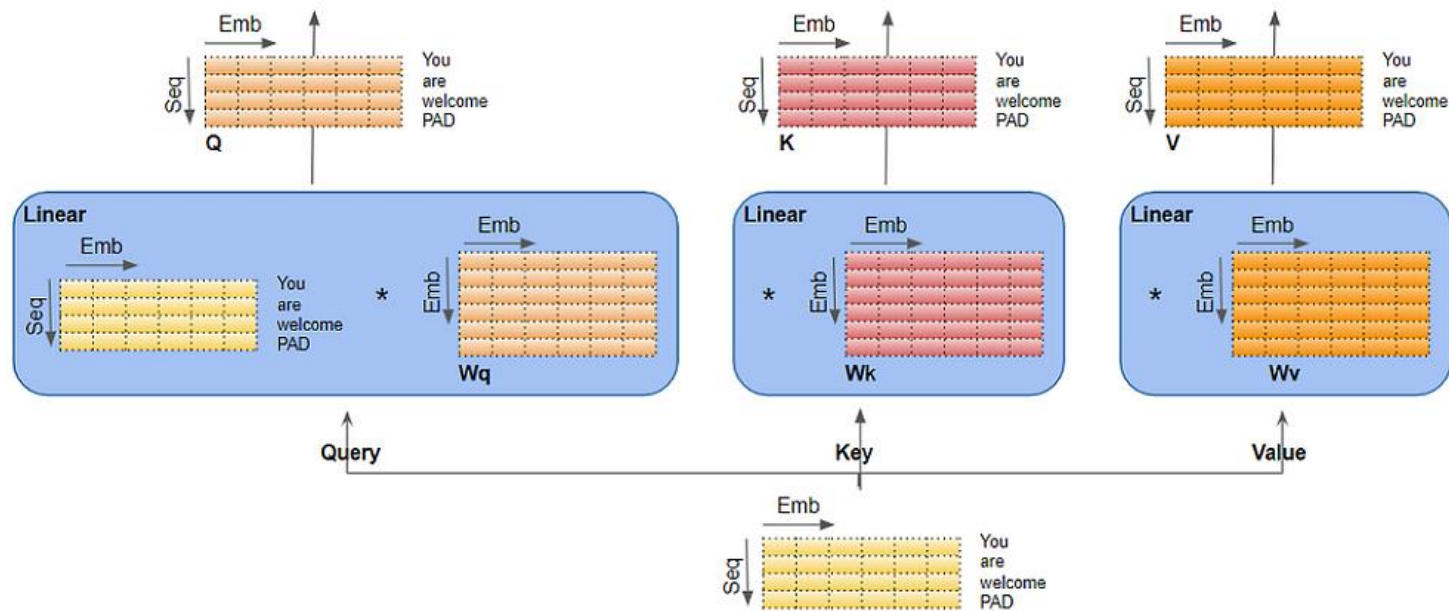
$$V_i = XW_i^V$$



Multi-head Attention – Linear Layers



Multi-head Attention – Linear Layers

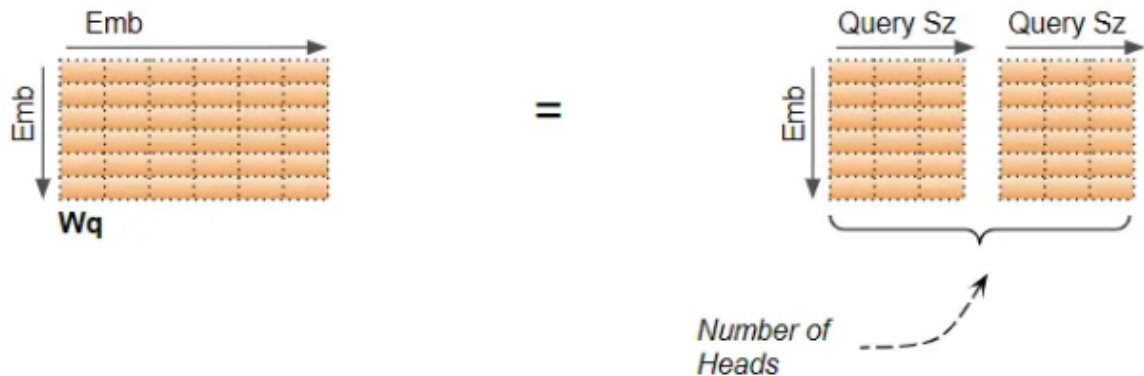


- **Independent Attention Computation:** Each head **independently** computes its own self-attention using the **scaled dot-product formula**.

Multi-head Attention – Linear Layers

Implementation in code:

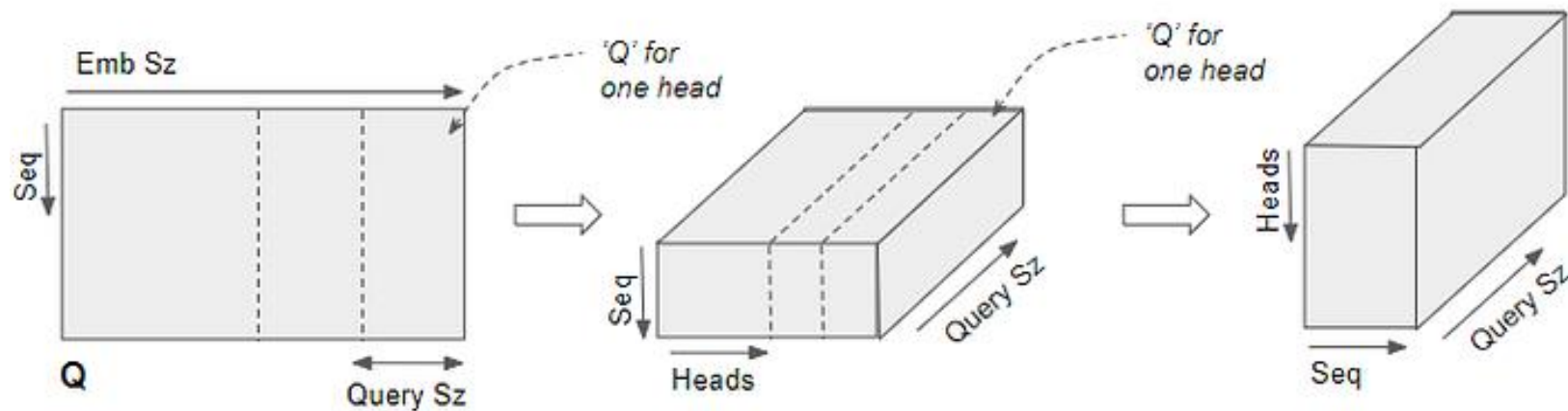
one bigger weight matrix that contains all heads' weights



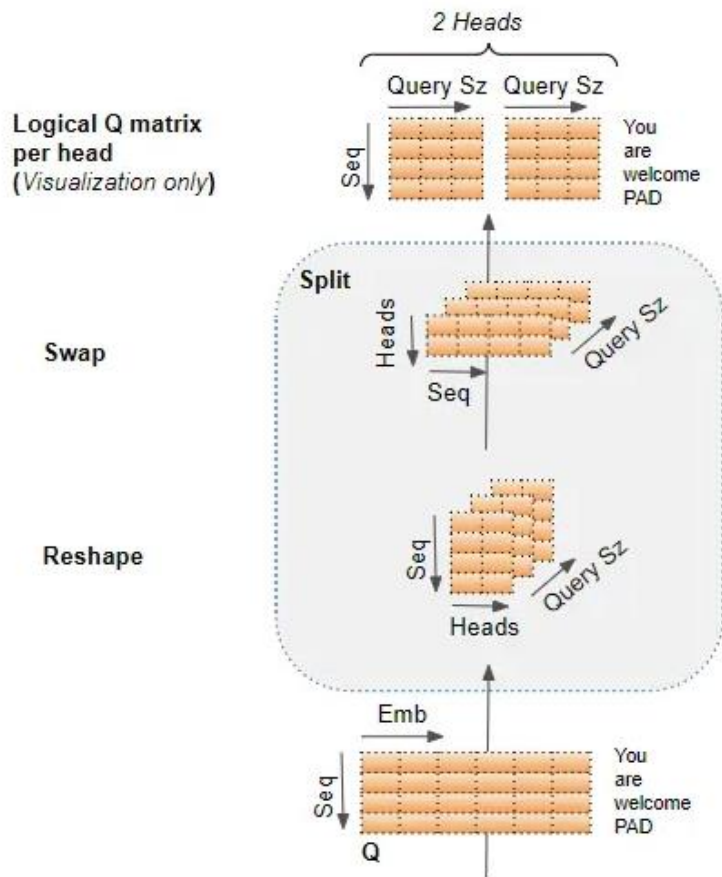
Multi-head Attention – Linear Layers

Split into heads!

- **Reshaping the Q, K, and V matrices**
- The Q, K, and V matrices output by the Linear layers are reshaped to include an explicit Head dimension.
- Each 'slice' corresponds to a matrix per head.

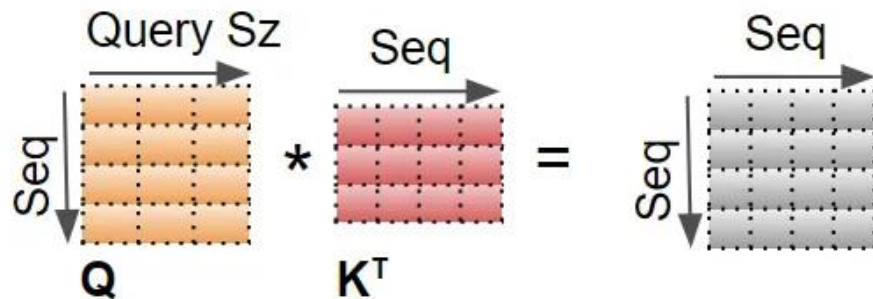


Compute Attention Score

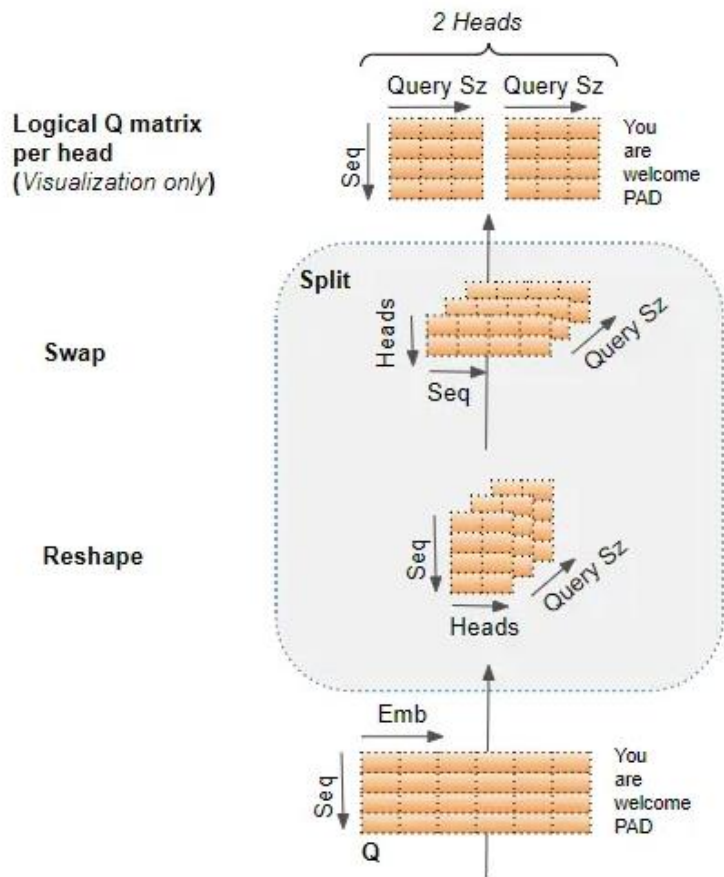


compute Q/K/V once, then split them into heads by reshaping.

Then compute scores **per head**



Compute Attention Score



compute Q/K/V once, then split them into heads by reshaping.

1. Packed Q,K,V: (B, N, D) where $D = h \cdot d_{head}$
2. Reshape ; Q ,K, V $\rightarrow (B, h, N, d_{head})$

Compute Attention Score

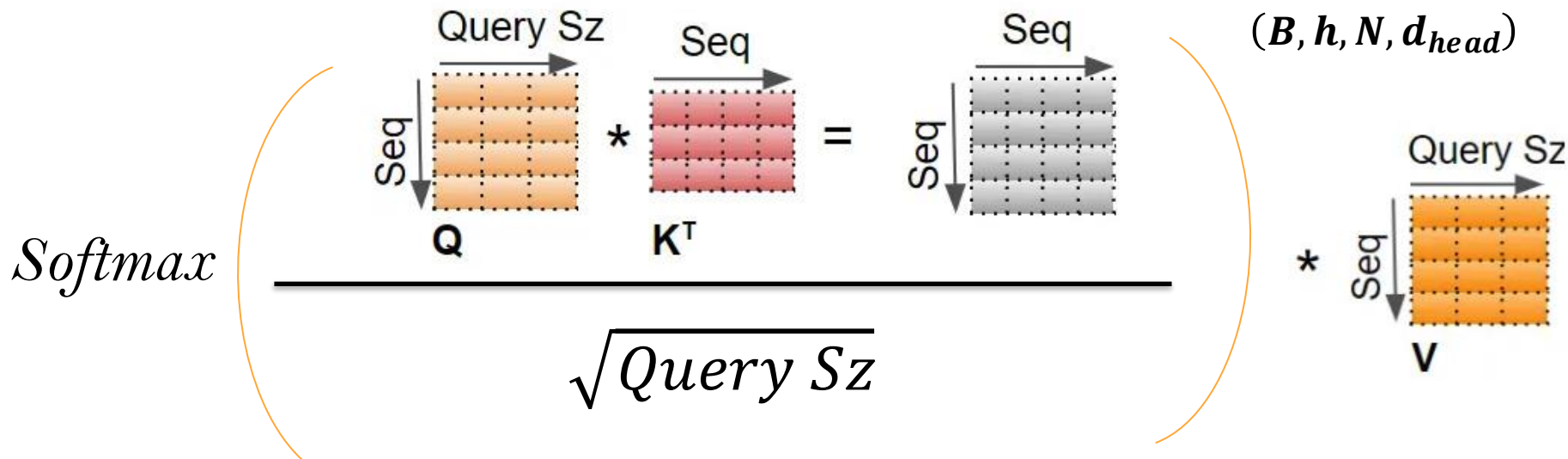
1. Packed Q,K,V: (B, N, D) where $D = h \cdot d_{head}$
2. Reshape ; Q ,K, V $\rightarrow (B, h, N, d_{head})$
3. Compute score (batched across heads)

$$Q @ K^T \rightarrow (B, h, N, N)$$

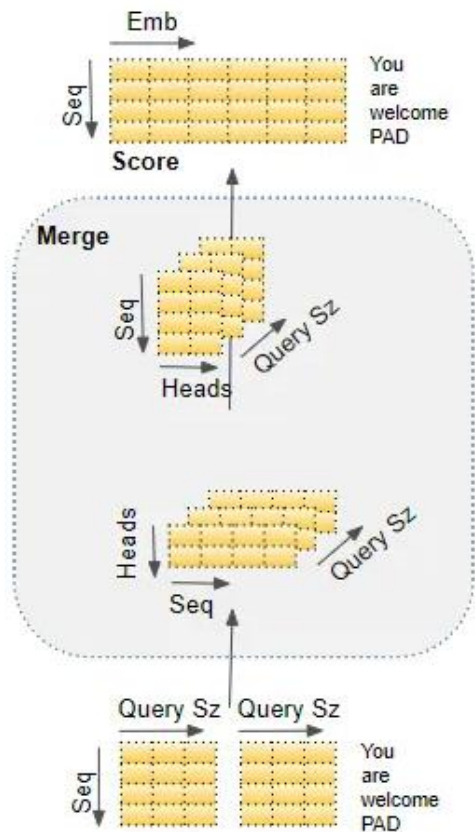
$$score = softmax(Q @ K^T) @ V$$



$$(B, h, N, d_{head})$$



Merge Each Head's Attention score



Reshape

$$MHA(X) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^0$$

(B, N, D)

(D, D)

where $D = h \cdot d_{\text{head}}$

Swap

W^0 **recombines information** across head and while keeping the same model dimension.

→ (Last linear layer weights)

Multi-head Attention – Linear Layers

```
class MultiHeadSelfAttention(nn.Module):
    def __init__(self, d_model: int, num_heads: int, bias: bool = True):
        super().__init__()
        assert d_model % num_heads == 0, "d_model must be divisible by num_heads"
        self.d_model = d_model
        self.num_heads = num_heads
        self.d_head = d_model // num_heads

        # One big linear for QKV (common efficient implementation):
        # produces [Q | K | V] concatenated on the last dimension
        self.qkv = nn.Linear(d_model, 3 * d_model, bias=bias)

        # Output projection
        self.out = nn.Linear(d_model, d_model, bias=bias)
```

```
def forward(self, x: torch.Tensor, attn_mask: torch.Tensor | None = None)
    """
    x: (batch, n, d_model)
    attn_mask (optional): broadcastable to (batch, heads, n, n)
                          e.g. causal mask with True=keep, False=block
    returns: (batch, n, d_model)
    """
    B, N, D = x.shape
    H, Dh = self.num_heads, self.d_head

    # 1) Compute one big QKV tensor in a single matmul
    qkv = self.qkv(x) # (B, N, 3*D)
    q, k, v = qkv.chunk(3, dim=-1) # each is (B, N, D)

    # 2) "Logical split": reshape D into (heads, d_head)
    q = q.view(B, N, H, Dh).transpose(1, 2) # (B, H, N, Dh)
    k = k.view(B, N, H, Dh).transpose(1, 2) # (B, H, N, Dh)
    v = v.view(B, N, H, Dh).transpose(1, 2) # (B, H, N, Dh)

    # 3) Scaled dot-product attention per head
    scores = (q @ k.transpose(-2, -1)) / math.sqrt(Dh) # (B, H, N, N)

    if attn_mask is not None:
        # If mask is boolean: True=allowed, False=blocked
        scores = scores.masked_fill(~attn_mask, float("-inf"))

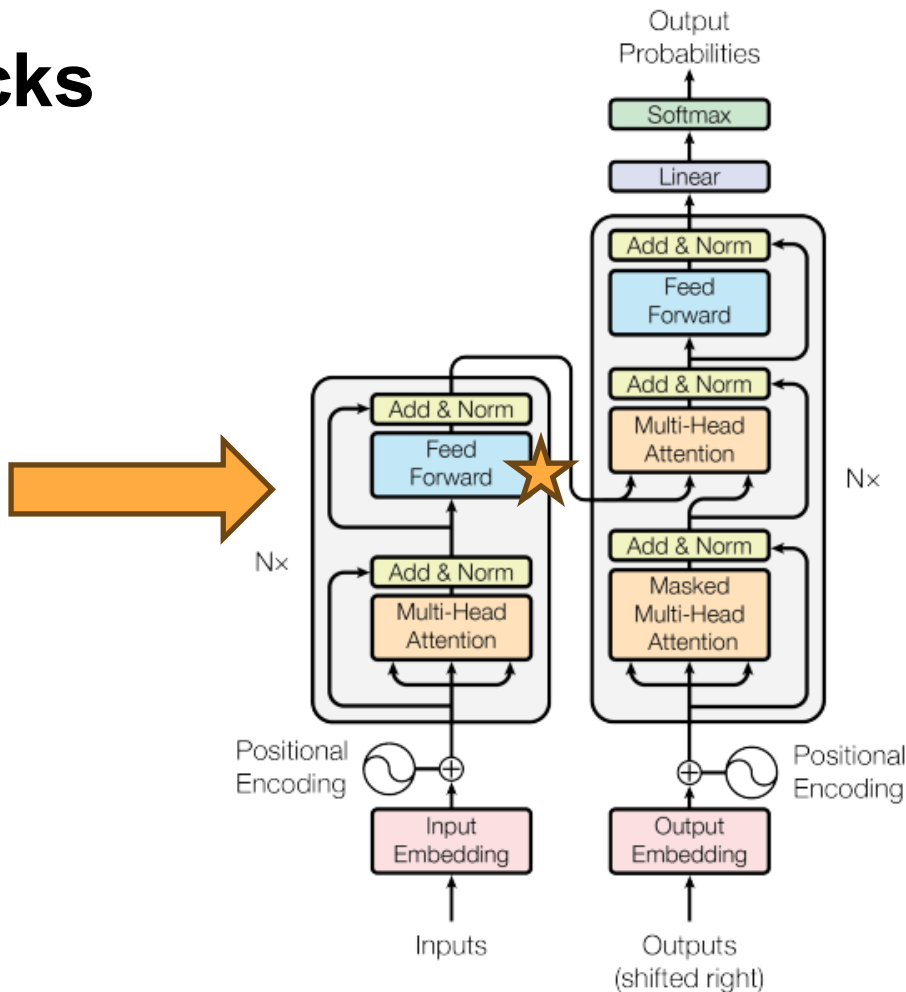
    attn = torch.softmax(scores, dim=-1) # (B, H, N, N)
    ctx = attn @ v # (B, H, N, Dh)

    # 4) Combine heads back: (B, N, H*Dh) == (B, N, D)
    ctx = ctx.transpose(1, 2).contiguous().view(B, N, D) # (B, N, D)

    # 5) Final linear projection
    return self.out(ctx)
```

Transformer Blocks

- Attention mixes **information across tokens.**
- The FFN mixes **features within each token.**
- Residual+Norm keep **training stable**



Residual connection: why it exists

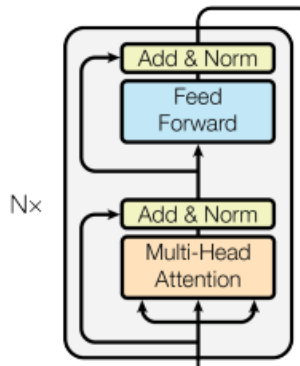
- Each layer in the transformer architecture consists of residual connections
- **Residual connections ensure that the information from the input of a layer is **preserved****

$$Y = X + f(X)$$

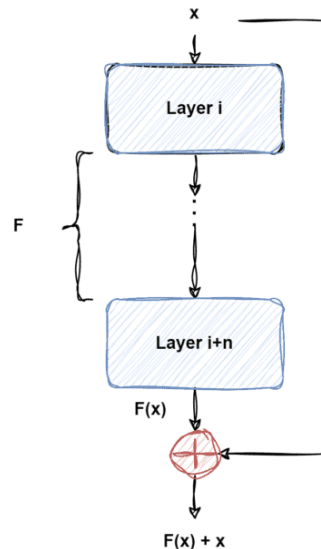
If $f(X)$ is small:
 $Y \approx X$

At start of training the weights can be small

Residual connections let each layer start as 'do nothing' and gradually learn 'do something useful'

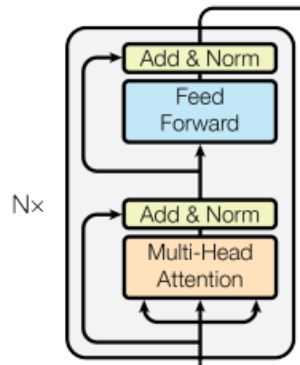


Feedforward with Residual Connection

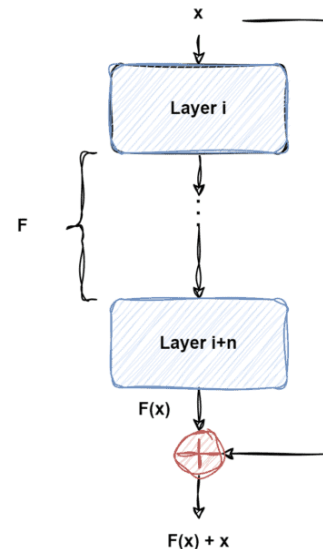


Residual connection: why it exists

- Maintaining **gradient flow** → solve the vanishing gradient problem
- **Optimizing** the training of deep transformer architectures → **enabling transformers to converge faster**



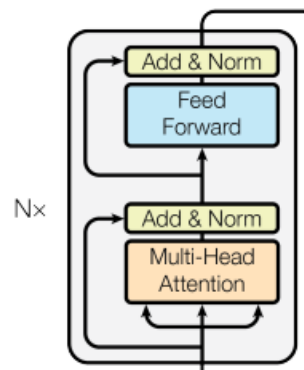
Feedforward with Residual Connection



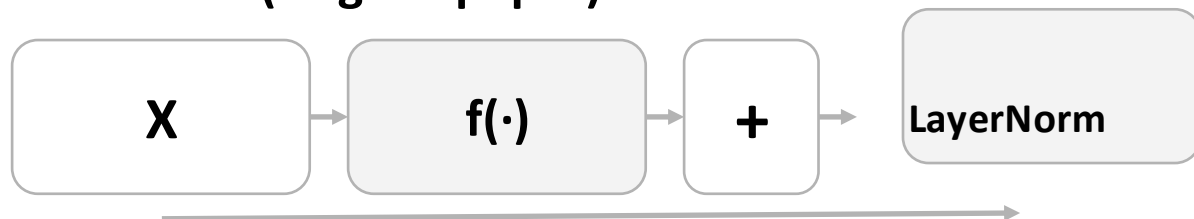
LayerNorm: why it exists

- Normalize across features per token → calculate mean and standard deviation for all features
- It **stabilizes training** by controlling activation/gradient scale.

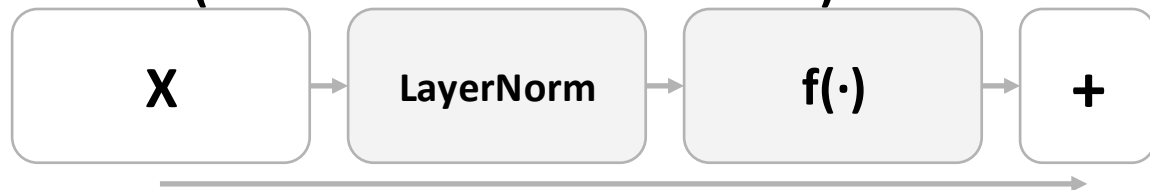
$$\hat{x} = \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} \quad y = \gamma \hat{x} + \beta$$



Post-LN (original paper)



Pre-LN (common in modern models)

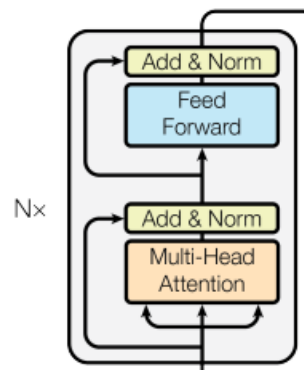


LayerNorm: why it exists

- Normalize across features per token → calculate mean and standard deviation for all features

$$\hat{x} = \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} \quad y = \gamma \hat{x} + \beta$$

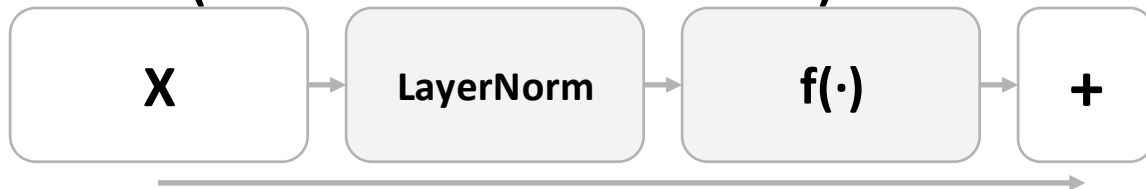
Post-LN (original paper)



Improves gradient stability in deep Transformers

- It **stabilizes** Keep $Y = X + \text{something}$ gradient easier to pass by controlling activation/gradient scale.

Pre-LN (common in modern models)



The Feedforward Network (FFN)

- **Fully Connected Layers**

- Two linear layers.

First layer: **expand the input dimension** from 512 to larger (2048)

Second layer: project **back** to model

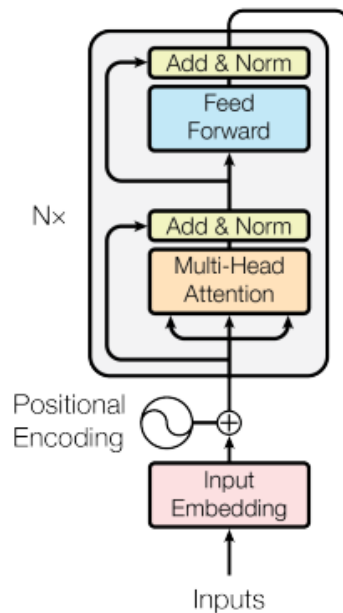
- **Active Function**

$\text{ReLU}(x) = \max(0, x)$

→ introduce non-linear into the model, help to learn more complex patterns

$$\text{FFN}(x) = W_2 \cdot \sigma(W_1 x + b_1) + b_2$$

Adds **nonlinearity** + **feature interaction**
(attention alone is mostly linear mixing)



Transformer Block

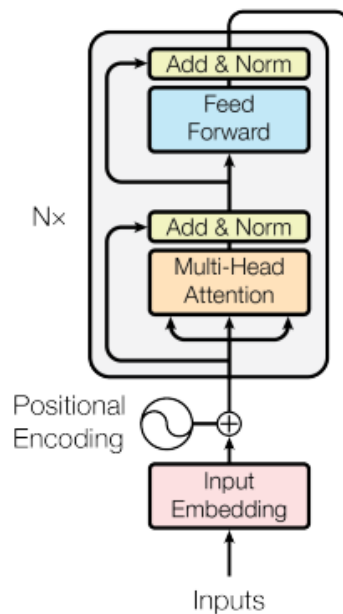
Two kinds of “mixing”

Self-attention

→ mixes across tokens (sequence axis N)

FFN / MLP

→ mixes across features (channel axis D)
(per token)



Transformer Block

I am a Large Language Model Researcher

1. Attention: Collects Evidence

Head 1: Token “researcher” attends strongly to Tokens “Large”, “Language”, “Model”

Head 2: Perdition linkage token “researcher” → token “am”

2. FFN: Compute new features from the collected evidences
activate a higher-level feature: profession = “LLM researcher”

Next steps

- Masking
Encoder Decoder
Cross-Attention

Additional Material

<https://poloclub.github.io/transformer-explainer/>

Thank you!

If you want to ask us any questions about the module, the university, or anything else, drop us an email.

Pro tip: write **COMM117** on the email subject line, to make sure we give it priority.



r.mu2@exeter.ac.uk

Questions?